

Práctica 3: Algoritmo de Tomasulo con Especulación: Etapa Commit

Arquitecturas de computación avanzadas

Curso 24/25

1. Objetivos de la práctica

- Comprender el funcionamiento de un esquema de planificación dinámica de instrucciones con especulación.
- Implementar y evaluar la etapa Commit del algoritmo de Tomasulo con especulación.
- Evaluar una *BTB* dentro del pipeline de MIPS que implementa el algoritmo de Tomasulo con especulación.

2. Desarrollo de la práctica

Para el desarrollo de la práctica se partirá de la versión del simulador *MARS-F* utilizada en la Práctica 2. Esta versión del simulador ya debería implementar la ruta de datos del algoritmo de Tomasulo sin especulación, a falta de implementar la etapa Commit.

La ruta de datos implementada difiere ligeramente de la presentada en las clases teóricas. Estructuralmente, la ruta de datos es idéntica a la vista en la Práctica 2 (Figura 1), aunque existen diferencias debido a la introducción de la ejecución especulativa. A continuación, se exponen las principales diferencias entre este diseño y el visto en clase:

- Aunque *MARS* dispone del conjunto de instrucciones completo de *MIPS*, la ruta de datos implementada sólo utiliza operaciones aritméticas con enteros. Para simular el comportamiento de Tomasulo, se supondrá que todas las operaciones de multiplicación y división de enteros se comportan como operaciones de punto flotante.
- Las operaciones de multiplicación y división siempre modifican los registros *HI* y *LOW*. Por esa razón, estas instrucciones siempre marcan esos registros en el fichero de registros, además de un tercer registro en las instrucciones de multiplicación de tres operandos¹.
- Si una estación de reserva está esperando alguno de sus operandos, puede comenzar la ejecución de su instrucción (si el operador está libre) en el mismo ciclo que lee el operando desde el CDB (etapa Writeback de la instrucción que produce el operando).
- La unidad de memoria difiere a la vista en clase. Por una parte, se disponen de dos unidades de cálculo de direcciones y de dos unidades de memoria: una para lecturas y otra para escrituras, permitiendo realizar una carga y un almacenamiento simultáneamente. Además, el cálculo de la dirección y la lectura/escritura se realizan en serie. Por tanto, en el caso de las operaciones de escritura, no se procederá al cálculo de la dirección hasta que se disponga tanto del registro de dirección como del registro a escribir.

¹Las operaciones de división con tres operandos en realidad son pseudo-instrucciones compuestas de una división con dos operandos seguida de una instrucción *mflo*.

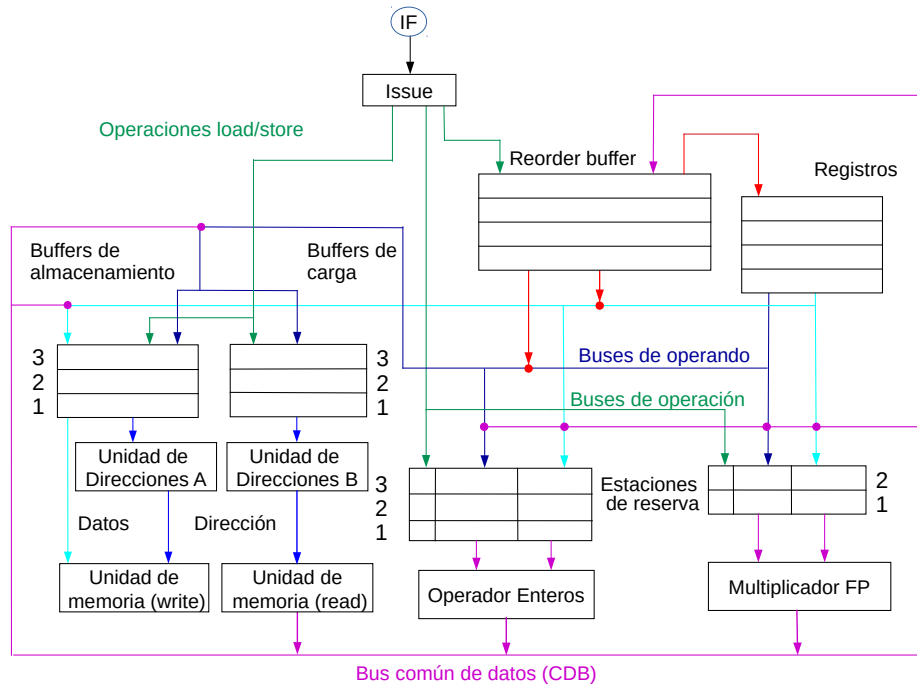


Figura 1: Ruta de datos del algoritmo de Tomasulo empleado en la Práctica 2.

En cuanto a las diferencias de la ruta de datos con la ruta de la Práctica 2:

- La nueva clase Tomasulo puede configurarse con los diferentes predictores de saltos del simulador, tanto estáticos como dinámicos (excepto *delayed branch*, ya que carece de sentido si se implementa planificación dinámica de instrucciones con especulación).
- En esta práctica se implementará la etapa Commit para poder especular correctamente. En caso de un fallo en la predicción del salto, se deberán liberar las entradas del RoB, los registros, los operadores y las estaciones de reserva. En las siguientes secciones se proporcionará el pseudocódigo de dicha etapa.
- **Las operaciones de almacenamiento en memoria son ejecutadas después de haber sido confirmadas.** Así se evita que instrucciones ejecutadas especulativamente de forma incorrecta modifiquen la memoria. Por ello, en los diagramas multiciclo se verá como una instrucción *sw* ejecuta las etapas *AC*, *M1* y *M2* después de la etapa *CO*.

2.1. Entrega de la práctica

La fecha límite de entrega será el lunes 28 de abril a las 23:55. Se deberá entregar:

- Memoria respondiendo a las preguntas hechas en el guion de prácticas.
- El código fuente de la clase `TomasuloEspeculacion_alumnos`.

El enunciado de la práctica se organiza como sigue: resumen de los principales paquetes y clases utilizadas en el simulador, la estructura de la unidad de gestión dinámica de instrucciones), el pseudocódigo de la etapa Commit del algoritmo de Tomasulo, y finalmente, ejercicios a realizar.

3. Estructura del simulador de Tomasulo

3.1. Paquetes y clases de java

La implementación del algoritmo de Tomasulo en *MARS-F* se compone de los siguientes paquetes y clases:

- Paquete `mars.pipeline`: Contiene diversas clases abstractas para la definición de los principales elementos del pipeline. Las clases que contiene son las siguientes:

- `Pipeline`: Clase abstracta para la definición de rutas de datos de procesador.
- `BranchPredictor`: Clase abstracta para la definición de predictores de saltos.
- `Stage`: Enumeración con las posibles etapas de la ruta de datos.
- `StageRegisters`: Registros inter-etapa. Se utilizan para que *MARS* se comunice con la ruta de datos de Tomasulo y para comunicar las etapas *IF* e *Issue*. Contienen la dirección y la instrucción a ejecutar, así como diversa información para la posterior ejecución de las instrucciones y visualización de los datos.
- `Decode`: Clase con múltiples métodos estáticos para decodificación de instrucciones. Por ejemplo:
 - `Decode.hasRs()` devuelve true si la instrucción tiene un primer operando, false en caso contrario.
 - `Decode.getRs()` devuelve el identificador del primer registro operando si la instrucción lo tiene, o cero en caso contrario.
 - `Decode.getDestination()` devuelve el identificador del registro destino si la instrucción lo tiene, o cero en caso contrario. La función devuelve siempre el registro correcto, sea *Rt* (instrucciones tipo I) o *Rd* (instrucción tipo R).
 - `Decode.getFormat()` devuelve el tipo de formato de la instrucción usando la clase *BasicInstructionFormat*, cuya salida puede ser:
 - ◊ `R_FORMAT`: instrucciones tipo R.
 - ◊ `I_FORMAT`: instrucciones tipo I con registro destino, es decir, loads, stores y aritmético-lógicas, las cuales poseen un registro operando (*Rs*), un inmediato y un registro destino (*Rt*).
 - ◊ `I_BRANCH_FORMAT`: instrucciones tipo I de bifurcación, las cuales poseen dos registros operandos (*Rs* y *Rt*) y no poseen registro de destino.
 - ◊ `J_FORMAT`: instrucciones tipo J.

Se recomienda encarecidamente leer la documentación de esta clase, ya que tiene todos los métodos necesarios para obtener la información de decodificación de una instrucción.

- `pipe_config`: Configura el pipeline a través de un fichero *xml*.
- Paquete `mars.pipeline.pipes`: Contiene la implementación de las rutas de datos sin planificación dinámica de instrucciones que fueron utilizadas en la práctica 1 (procesador *MIPS* segmentado de 5 etapas y con unidades multiciclo).
- Paquete `mars.pipeline.predictors`: Contiene la implementación de diversos predictores de saltos:
 - `IdealPredictor`: Predictor ideal que siempre acierta.
 - `StopPredictor`: Predictor que soluciona los riesgos de control deteniendo la ejecución (en realidad no predice nada).
 - `NotTakenPredictor`: Predice siempre no tomado.
 - `TakenPredictor`: Predice siempre tomado.

- `DynamicPredictor`: Clase abstracta para implementar predictores dinámicos.
 - `BitPredictor`: Implementa una “*Branch Prediction Table*” con predictores dinámicos de 1, 2 o 3 bits.
 - `BTB`: Implementa un predictor dinámico conocido como “*Branch Target Buffer*” (*BTB*). El funcionamiento de la *BTB* se repasará más adelante de cara a su utilización en el práctica 3.
- Paquete `mars.pipeline.tomasulo`: Contienen las clases que implementan la ruta de datos con el Algoritmo de Tomasulo.
- Clases necesarias para la realización de esta práctica:
 - `CDB`: Implementa el buffer común de datos o CDB. Ofrece métodos estáticos para acceder a la única instancia del CDB.
 - `registro`: Implementa el banco de registros. Ofrece métodos estáticos para acceder a la única instancia del banco de registros.
 - `Estacion`: Clase abstracta para definir estaciones de reserva. Contiene dos métodos abstractos a implementar en cada subclase: *Execute()* y *Ocupar()*. *Execute()* ejecuta un ciclo de la instrucción asociada a la estación de reserva y ya está implementado. *Ocupar()* guarda una instrucción en una estación de reserva libre. Las implementaciones de los métodos *Ocupar()* en las subclases de `Estacion` serán realizadas durante esta práctica. También habrá que implementar los métodos *checkQj()* y *checkQk*, que servirán para comprobar la disponibilidad de los operadores de cada instrucción.
 - `ReorderBuffer`: Clase que implementa la estructura de datos y métodos para manejar el buffer de reordenamiento o “*Reorder Buffer*” (*rob*). Ofrece un método estático para acceder a la única instancia del *rob*.
 - `RobEntry`: Clase que implementa la estructura de datos y métodos para manejar una entrada del *rob*.
 - `Tomasulo`: Clase abstracta que define los métodos para la ejecución de la ruta de datos de Tomasulo e implementa las estructuras de datos necesarias para su funcionamiento.
 - `Tomasulo_conf`: Contiene todos los parámetros de configuración de la ruta de datos Tomasulo, como la latencia de los operadores, número de estaciones de reserva de cada tipo, etc.
 - `TomasuloIdeal_alumnos`: Subclase que implementa el algoritmo de Tomasulo que se utilizará en la práctica 2. Todos los métodos están implementados excepto los métodos *Issue()* y *Writeback()*, que serán implementados durante esta práctica.
 - `TomasuloEspeculacion_alumnos`: Subclase que implementa el algoritmo de Tomasulo con especulación que se utilizará en la práctica 3. Todos los métodos están implementados, excepto los métodos *Issue()* y *Writeback()*, que serán copiados de la clase *TomasuloIdeal_alumnos*, y el método *Commit()*, que será implementado durante esa práctica.
 - Otras clases necesarias para la realización de esta práctica pero con las que no será necesario interactuar durante el desarrollo de la misma:
 - `Tomasulo_alumnos`: Subclase de `Tomasulo` que declara las estaciones de servicio utilizando las subclases a implementar por los alumnos. Su única misión es establecer una jerarquía de clases para distinguir entre las clases a implementar y las ya implementadas (no disponibles en esta versión del simulador pero si en la versión completa sin código fuente).
 - `InstructionSetTomasulo`: Implementa el set de instrucciones *MIPS* que soporta nuestra ruta de datos Tomasulo.

- **Operador:** Implementa un operador genérico. Como en realidad la ejecución de las instrucciones la realiza la clase `InstructionSetTomasulo`, basta con un único operador para todos los tipos de operación.
- **Paquete `mars.pipeline.tomasulo.DiagramaMulticiclo`:** Incluye clases para la gestión del diagrama de ejecución multiciclo.
 - `DiagramaMulticiclo`: Clase que implementa el diagrama de ejecución multiciclo.
 - `InstructionInfo`: Clase que guarda diversa información de las instrucciones que se utilizará para la posterior visualización del diagrama multiciclo.
 - `HtmlUtils`: Clase con múltiples métodos estáticos para la generación de estructuras html utilizadas para generar la información de profiling de la ruta de datos.
- **Paquete `mars.pipeline.tomasulo.estaciones_alumnos`:** Contiene las subclases de `Estacion` que implementan las diversas estaciones de reservas utilizadas:
 - `EstacionINT_alumnos`: clase para estaciones de reserva de operaciones de enteros.
 - `EstacionPF_alumnos`: clase para estaciones de reserva de operaciones de punto flotante. Además de los métodos heredados, posee nuevos métodos para gestionar los registros HI y LOW.
 - `EstacionSW_alumnos`: clase para estaciones de reserva de operaciones de instrucciones de almacenamiento en memoria (*sw*).
 - `EstacionLW_alumnos`: clase para estaciones de reserva de operaciones de instrucciones de carga desde memoria (*lw*).

Todas las clases necesarias para la realización de la práctica incluyen la documentación en formato JavaDOC. En el Campus Virtual de la asignatura se puede descargar toda la documentación generada por el proyecto. También puede generarse desde el IDE de programación utilizado. Notar que la documentación relevante para la realización de la práctica será únicamente la incluida en el paquete `mars.pipeline`.

3.2. Estructura de la unidad de gestión dinámica de instrucciones (clase Tomasulo)

La unidad de gestión dinámica está compuesta por los siguientes elementos:

- **Estaciones de Reserva de operaciones de enteros.** Contiene las estaciones de reserva del operador de enteros. Está representada por la variable `est_Enter`. El número de estaciones de reserva viene indicado por la constante `NUM_EST_ENTEROS`² (por defecto, 4).
- **Operador Enteros.** Se encarga de realizar las operaciones de enteros. Está representado por la variable `opEnter`. La latencia en ciclos del operador viene indicada por la constante `LAT_ENTEROS` (por defecto, 1).
- **Estaciones de Reserva de operaciones de punto flotante.** Contiene las estaciones de reserva del operador de punto flotante. Está representada por la variable `est_PF` y la constante `NUM_EST_PF` indica el número de estaciones de reserva (por defecto, 2).
- **Operador de coma flotante.** Se encarga de realizar las operaciones de coma flotante. Está representado por la variable `opPF`. La latencia en ciclos del operador viene indicada por la constante `LAT_PF` (por defecto, 6). El operador no está segmentado.
- **Buffers de escritura o almacenamiento.** Contiene las estaciones de reserva para las operaciones de escritura en memoria. Está representada por la variable `est_Store`. El número de buffers viene indicado por la constante `NUM_EST_STORE` (por defecto, 3).

²Las constantes indicadas a lo largo de esta sección se encuentran en la clase `Tomasulo_conf`.

- **Unidad de escritura de memoria.** Se encarga de realizar las operaciones de almacenamiento en memoria. Está representado por la variable `opStore`. La latencia en ciclos de la unidad viene indicada por la constante `LAT_STORE` (por defecto, 3 ciclos: 1 para el cálculo de la dirección y 2 para la escritura en memoria). El operador no está segmentado.
- **Buffers de lectura o carga.** Contiene las estaciones de reserva para las operaciones de lectura en memoria. Está representada por la variable `est_Load`. El número de buffers viene indicado por la constante `NUM_EST_LOAD` (por defecto, 3).
- **Operador de lectura de memoria.** Se encarga de realizar las operaciones de lectura de memoria. Está representado por la variable `opLoad`. La latencia en ciclos de la unidad viene indicada por la constante `LAT_LOAD` (por defecto, 3 ciclos: 1 para el cálculo de la dirección y 2 para la escritura en memoria). El operador no está segmentado.
- Además, existe una variable llamada `est_Todas` que contiene todas las estaciones de reserva. Puede ser de utilidad si se desea recorrer todas las estaciones para realizar una tarea común a todas, independientemente de la subclase.
- **Buffer de Reordenamiento o Reorder Buffer.** Se encarga de la gestión del *rob* y está representado por la variable `rob`. Sin embargo, se puede acceder al *rob* desde cualquier clase a través del método estático `ReorderBuffer.getRoBInstance`³. El número de entradas del Reorder Buffer viene indicado por la constante `NUM_ENTRADAS_ROB` (por defecto, 20).

Además, aunque no son variables de la clase *Tomasulo*, caben destacar:

- **Bus común de datos o CDB:** Se encarga de las transferencias entre los diversos componentes del sistema. Se puede acceder al CDB desde cualquier clase a través del método estático `CDB.read()`.
- **Banco de registros.** Se puede acceder al banco de registros desde cualquier clase a través del método estático `registro.getRegs()` o a un solo registro a través del método estático `registro.getReg()`. Además, podemos acceder a las constantes `NUM_REGISTROS` (número de registros), `HI` (identificador registro `$hi`) y `LOW` (identificador registro `$low`).

Por último, señalar que la clase *Tomasulo.conf* contiene la constante `ES_NULO`, que nos servirá para marcar las etiquetas de estaciones, operadores y registros cuando no estén en uso o estén esperando una actualización.

3.3. Preparación de la práctica

Antes de comenzar la implementación de la etapa *Commit*, se deben sustituir los métodos *Issue()* y *Writeback()* de la clase *TomasuloEspeculacion_alumnos* por los implementados durante la Práctica 2 en la clase *TomasuloIdeal_alumnos*.

³En realidad, la variable `rob` se obtiene a través de método `ReorderBuffer.getRoBInstance` durante la construcción del objeto de la clase *Tomasulo*.

3.4. Pseudocódigo de la Etapa Commit

A continuación se muestra el pseudocódigo de la etapa *Commit* del algoritmo de Tomasulo con especulación:

```

/* Obtenemos la primera entrada del Reorder Buffer
   y la instrucción almacenada en la misma */
EntradaRob head := rob.getHead()
Instrucción I := head.instruccion

/* Comprobamos el tipo de instrucción.
   Las tareas a realizar dependerán de si tenemos un almacenamiento,
   un salto condicional u otro tipo de instrucción (entero, pf o carga) */

Si head.EsSalto; entonces

    // Comprobamos el resultado del salto y su predicción
    Si head.resultadoSalto == head.predicción; entonces
        // predicción correcta, actualizamos el contador de aciertos
        aciertos_predictor++
    Sino
        /* Se ha fallado la predicción. Se debe descartar todo
           el trabajo especulado incorrectamente.
           ATENCIÓN: no eliminar esta instrucción del código*/
        emitiendoIncorrectas = false
        // actualizamos el contador de fallos
        fallos_predictor++

        /* Comenzamos liberando los registros. Es decir,
           la etiqueta Q_i de cada registro toma el valor ES_NULO */
        Para (todos los Registros reg) hacer
            reg.libera()

        /* Liberamos las estaciones de reserva de enteros, pf y carga.
           En cuanto a las estaciones de reserva de almacenamiento,
           hay que comprobar si la estación está confirmada.
           En ese caso, no se libera para completar su ejecución,
           ya que la instrucción se emitió correctamente*/
        Para (todas las Estaciones de reserva estacion) hacer
            Si estacion es una instancia de EstacionSW; entonces
                Si estacion.confirmada == false; entonces
                    estacion.libera()
            Sino
                estacion.libera()

        /* Por la misma razón, liberamos todos los operadores
           excepto el operador de almacenamiento */
        operador_enteros.libera()
        operador_pf.libera()
        operador_carga.libera()

        /* Eliminamos las instrucciones en los registro IF e ISSUE.
           Para ello, se escribe una nop, que se codifica como 0 */
        registro_interEtapa[0].Instruccion := 0
        registro_interEtapa[1].Instruccion := 0

```

```

        /* Limpiamos todas las instrucciones del ROB*/
        rob.clean()
Sino Si I.tipo == almacenamiento; entonces
    /* Obtenemos la estación de reserva y confirmamos la operación.
       Así el operador podrá escribir el resultado en memoria */
    Estacion estacion := head.getEstacion()
    estacion.confirma := true

Sino /* En este caso, es una operación de enteros, de pf o almacenamiento.
      Si la instrucción modifica un registro, actualizamos su valor*/
    Si head.destino !=0; entonces
        Registro reg = registro[head.destino]
        reg.valor := head.resultado

        /* Si la marca del registro es el identificador
           de la entrada actual del rob, liberamos el registro */
        Si reg.Qi == head.ID; entonces
            reg.libera()

        /* CASO ESPECIAL. Para punto flotante debemos actualizar
           los registros HI y LOW*/
        Si I.tipo == pf; entonces
            registro[HI].valor := head.HI
            registro[LOW].valor := head.LOW

        Si registro[HI].Qi == head.ID; entonces
            registro[HI].libera()
            registro[LOW].libera()
/* Finalmente, eliminamos la entrada de la instrucción del RoB */
rob.removeHEAD()

```

4. Ejercicios a realizar

- 1 Implementación del algoritmo de Tomasulo con especulación. Para ello, se añadirán los métodos `Issue()` y `WB()` implementados en la Práctica 2 al fichero `TomasuloEspeculacion-alumnos.java` y se implementará el método `Commit()`.
- 2 Comprobar el correcto funcionamiento del algoritmo de Tomasulo con especulación. Para ejecutar la versión de Tomasulo de esta práctica con la configuración por defecto, basta con crear un fichero xml con el siguiente contenido:

```

<?xml version="1.0" encoding="utf-8"?>
<pipeline tipo="tomasulo" especulacion="si" limite="100" ventana="25">
    <!-- predictores aceptados: ideal, taken y nottaken-->
    <predictor>taken</predictor>
</pipeline>

```

El fichero `TomasuloPredictorEstatico.xml` de los ejemplos proporcionados en el Campus Virtual contiene todos los parámetros configurables de este pipeline.

Para comprobar el funcionamiento del pipeline, utilizaremos el siguiente código que calcula el mínimo común divisor de dos números naturales m y n disponible en el Campus Virtual con el nombre de `mcd.asm`⁴:

⁴El código de ejemplo contiene un salto incondicional j . Se asume predicción ideal de saltos incondicionales (j ,


```

.data
S: .space 4
.text
#este programa determina el máximo común divisor
#usando el algortimo de euclides
#para dos números naturales a y b
li $t0, 607152      # a
li $t1, 1390        # b
slt $t2, $t0, $t1   # a < b?
beq $t2, $zero, loop
#si b > a, intercambiamos los valores
move $t2, $t1       # aux = b
move $t1, $t0       # b=a
move $t0, $t2       # b = aux
loop:
    beq $t1, $zero, save_result
    div $t0, $t1
    move $t0, $t1    # a=b
    mfhi $t1         # b= a mod b
    j loop
save_result:
la $t3, S           #cargamos dir. de guardado
sw $t0, 0($t3)      #guardamos mcd
li $v0, 10
syscall

```

Se invocará la ejecución del simulador utilizando la siguiente sintaxis:

```
java -jar MARS-F.jar pipe mi_pipe_file.xml ejemploP3.asm
```

Al ejecutar el simulador se generarán diversos ficheros en formato html por cada ciclo con la información sobre el estado de la máquina y el diagrama de ejecución multiciclo, además de un resumen con las instrucciones ejecutadas y su tipo. Para visualizar los ficheros se puede utilizar cualquier navegador.

Para comprobar el correcto funcionamiento del algoritmo, se puede utilizar la versión del simulador con el algoritmo de Tomasulo implementado suministrada en el Campus Virtual de la asignatura.

Indica cuál es el tiempo de ejecución total (en ciclos), CPI, las instrucciones totales y las instrucciones especuladas incorrectamente para el programa `mcd.asm` utilizando ambos predictores.

Predictor	<i>Predict taken</i>	<i>Predict not taken</i>
Ciclos ejecutados:		
Total de instrucciones ejecutadas:		
Instrucciones mal especuladas:		
CPI:		

- Comprobar el funcionamiento de la operación SDOT ($k = \vec{X} \cdot \vec{Y}$). Para ello usaremos el fichero `sdot30.asm`, ya utilizado en la práctica 2, modificando el procesador para tener 4 estaciones de reserva de punto flotante y 4 buffers de carga, manteniendo el número de buffers de almacenamiento y de estaciones de reserva de enteros en su valor por defecto.

jal y *jr*), por lo que tras un salto incondicional, en el siguiente ciclo se lanzará la instrucción destino del salto, tratando la instrucción *j* como una instrucción de enteros más y pasando ésta por las etapas *Is*, *EX*, *WB* y *CO*.

Indica cuál es el tiempo de ejecución total (en ciclos), CPI, las instrucciones totales y las instrucciones especuladas incorrectamente para el programa `sdot30.asm` utilizando ambos predictores. Indica en observaciones la diferencia de rendimiento entre ambas configuraciones y el motivo de esta diferencia.

Predictor	<i>Predict taken</i>	<i>Predict not taken</i>
Ciclos ejecutados:		
Total de instrucciones ejecutadas:		
Instrucciones mal especuladas:		
CPI:		
Observaciones:		

4 A continuación, comprobaremos el funcionamiento del procesador con planificación dinámica y especulación junto a una predictor dinámico. En concreto, utilizaremos una *BTB* con las siguientes características:

- Tabla totalmente asociativa.
- Dos entradas por defecto.
- Predictor de 2 bits por entrada en la BTB.
- Algoritmo de reemplazo *LRU* (*Least Recently Used*).

Para estudiar el comportamiento de la *BTB*, utilizaremos el siguiente código, el cuál ordena un vector de 19 enteros utilizando el método de selección:

```
.data
vector:    .word 7,18,5,14,15,1,12,16,13,9,11,2,10,8,0,6,17,4,3
Orden:     .word 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18
Desorden:  .word 18,17,16,15,14,13,12,11,10,9,8,7,6,5,4,3,2,1,0
.text
la $t0, vector                #carga el vector
addi $t1, $t0, 72             #límite i (dirección V[17])
addi $t2, $t0, 76             #límite j (dirección V[18])
move $t3, $t0                 #dirección V[i]
loopI:
    lw $t5, 0($t3)             #carga V[i]
    addi $t4, $t3, 4           #dirección inicial
                                #V[j]= dirección V[i+1]
loopJ:
    lw $t6, 0($t4)             #carga V[j]
    slt $t7, $t6, $t5          # V[j] < V[i]?
    beq $t7, $zero, incJ       #si V[j] > V[i], salto
    sw $t5, 0($t4)             #si no, guardo V[j] en
                                #la posición de V[i]
    move $t5, $t6              #actualizo el registro que
                                #antes contenía V[i]
incJ:
    addi $t4, $t4, 4           #incremento V[j]
    bne $t4, $t2, loopJ        #compruebo si se ha llegado
                                #al final en el loop J
    sw $t5, 0($t3)             #guardo el valor de V[i]
    addi $t3, $t3, 4           #incremento V[i]
    bne $t3, $t1, loopI
end:
    li $v0, 10
    syscall
```

Para ejecutar el pipeline de Tomasulo con un predictor dinámico como la *BTB*, tendremos que indicar el predictor deseado en el fichero de configuración, indicando también el número de entradas de nuestro predictor dinámico. Por ejemplo, para ejecutar Tomasulo con una BPB de predictores de 2 bits y 8 entradas, el fichero de configuración sería el siguiente:

```
<?xml version="1.0" encoding="utf-8"?>
<pipeline tipo="tomasulo" especulacion="si" limite="100" ventana="25">
  <predictor>bit2</predictor>
  <predictor_entries>8</predictor_entries>
</pipeline>
```

El fichero `TomasuloPredictorDinamico.xml` de los ejemplos proporcionados en el Campus Virtual contiene todos los parámetros configurables de este pipeline, mientras que el fichero `ordena19.asm` incluido con la Práctica 3 contiene el código mostrado anteriormente.

Se invocará la ejecución del simulador utilizando la siguiente sintaxis:

```
java -jar MARS-F.jar pipes mi_pipe_file.xml ordena19.asm
```

Esta orden generará ficheros en formato `html` por cada ciclo con la información sobre el estado de la máquina, el diagrama de ejecución multiciclo y un resumen con las instrucciones ejecutadas y su tipo, como en las anteriores prácticas. Además, también se crearán múltiples ficheros `html` mostrando el contenido de la *BTB*. A diferencia de los ficheros de estado y de diagrama multiciclo, que se imprimen ciclo a ciclo durante los primeros 100 ciclos, los ficheros *BTB* se generan solamente en los ciclos en los que se modifica la *BTB* y durante toda la duración de la simulación.

Indica cuál es el tiempo de ejecución total (en ciclos), CPI, las instrucciones totales y número de saltos predichos correctamente para el programa `ordena19.asm` utilizando la *BTB*.

Ciclos ejecutados:

Total de instrucciones ejecutadas:

Saltos predichos correctamente:

CPI:

- 5 Aumenta el número de entradas de la *BTB* a 4. Vuelve a ejecutar el programa `ordena19.asm` e indica cuál es el tiempo de ejecución total (en ciclos), CPI, las instrucciones totales, número de saltos predichos correctamente y el incremento del rendimiento. Explica porque varía el rendimiento de este programa al aumentar las entradas en la *BTB*.

Ciclos ejecutados:

Total de instrucciones ejecutadas:

Saltos predichos correctamente:

CPI:

Incremento del rendimiento (%):

Explicaciones:

- 6 El fichero `ordena19.asm` contiene tres vectores diferentes en la sección de datos: uno con los valores ordenados al azar, un segundo vector con sus elementos complemente ordenados y un tercer vector con sus elementos complemente desordenados. Ejecuta el programa para ordenar los tres vectores usando los predictor *predict not taken* y *predict taken*. Utiliza para ello el pipeline de la Práctica 3 con su configuración por defecto. Anota el tiempo de ejecución en ciclos y el número de saltos predichos correctamente. Después ordena los tres vectores utilizando la *BTB* de cuatro entradas y anota el tiempo de ejecución en ciclos, el número de saltos predichos correctamente y el incremento del rendimiento respecto a su ejecución utilizando el predictor *predict taken*. Razona el motivo por el cual varía el comportamiento del programa en función del vector y del tipo de predictor.

Vector	<i>Aleatorio</i>	<i>Ordenado</i>	<i>Desordenado</i>
T. ejecución <i>notTaken</i> (ciclos):			
Predicciones correctas <i>notTaken</i>:			
T. ejecución <i>taken</i> (ciclos):			
Predicciones correctas <i>taken</i>:			
T. ejecución <i>BTB</i> (ciclos):			
Predicciones correctas <i>BTB</i>:			
Speed-Up <i>BTB/notTaken</i> (%):			
Speed-Up <i>BTB/taken</i> (%):			
Explicaciones:			